PRÁTICA LABORATORIAL 06

Objetivos:

- Compreender o que é o SASS
- Instalar e configurar o SASS (SCSS)
- Utilizar variáveis
- Criar aninhamento de seletores
- Usar partials e imports
- Criar mixins
- Utilizar extends
- Compilar SCSS para CSS

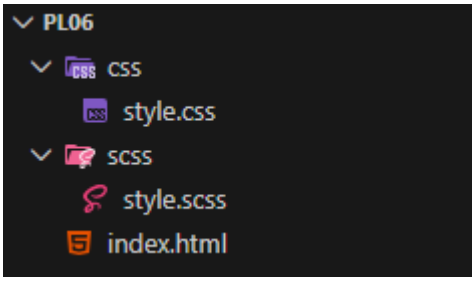
SASS (Syntactically Awesome Style Sheets) é um pré-processador de CSS, ou seja, uma linguagem que estende o CSS com funcionalidades que não existem nativamente.

O browser NÃO interpreta SASS. O SASS é convertido em CSS normal

Existem duas sintaxes:
.sass (sintaxe antiga, sem chavetas)
.scss (mais usada, semelhante ao CSS)

PARTE I – CRIAÇÃO DO PROJETO

Crie uma pasta PL06 com um ficheiro index.html, uma pasta css com o ficheiro style.css e uma pasta scss com um ficheiro style.scss.



No ficheiro index.html ligar o ficheiro CSS: <link rel="stylesheet" href="css/style.css">
Importante, nunca ligamos o ficheiro SCSS diretamente ao HTML.

```html
index.html > html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>

</body>
</html>
```

PARTE II – INSTALAÇÃO E INICIALIZAÇÃO DO SASS

No terminal executar o comando: npm install -g sass
Depois testar a instalação validando a versão instalada com: sass –version

```
PS C:\Users\bruno\Desktop\PL06> npm install -g sass

changed 12 packages in 893ms

5 packages are looking for funding
  run `npm fund` for details
PS C:\Users\bruno\Desktop\PL06> sass --version
1.97.3 compiled with dart2js 3.10.7
PS C:\Users\bruno\Desktop\PL06>
```

Para que a compilação esteja sempre a ser executada vamos correr o comando: sass --watch scss/style.scss css/style.css

```
PS C:\Users\bruno\Desktop\PL06> sass --watch scss/style.scss css/style.css
Sass is watching for changes. Press Ctrl-C to stop.
```

Agora sempre que alterarmos o ficheiro SCSS este irá transpor as alterações para o CSS.

```scss
scss > style.scss > ...
$cor-primaria: #3498db;
$cor-secundaria: #2ecc71;
$tamanho-base: 16px;

body {
  font-size: $tamanho-base;
  background-color: $cor-primaria;
  color: white;
}
```

```css
css > style.css > ...
body {
  font-size: 16px;
  background-color: #3498db;
  color: white;
}

/*# sourceMappingURL=style.css.map */
```

PARTE III – NESTING

O nesting, ou aninhamento de seletores, é uma funcionalidade do SCSS que permite escrever seletores dentro de outros seletores, seguindo a mesma hierarquia do HTML.

Em CSS tradicional, cada seletor tem de ser escrito de forma independente, o que pode levar à repetição de código e a ficheiros difíceis de ler e manter. Com o nesting, o SCSS permite agrupar estilos relacionados, tornando o código:

- Mais organizado
- Mais legível
- Mais fácil de manter
- Mais próximo da estrutura do HTML

O aninhamento é especialmente útil em elementos como menus de navegação, headers, cards e componentes, onde existe uma relação clara entre elementos pai e filhos.
Apesar de ser uma grande vantagem, o nesting deve ser usado com moderação. Um excesso de níveis aninhados pode gerar seletores CSS demasiado longos e difíceis de gerir.

No exemplo seguinte será possível observar como o SCSS permite aplicar estilos a elementos internos de forma clara e estruturada, sem repetir o seletor principal várias vezes:

scss > style.scss > ...

```
5   body {
6     font-size: $tamanho-base;
7     background-color: $cor-primaria;
8     color: white;
9   }
10
11  nav {
12    background-color: #333;
13
14    ul {
15      list-style: none;
16
17      li {
18        display: inline-block;
19
20        a {
21          color: white;
22          text-decoration: none;
23        }
24      }
25    }
26  }
27
```

css > style.css > ...

```
1   body {
2     font-size: 16px;
3     background-color: #3498db;
4     color: white;
5   }
6
7   nav {
8     background-color: #333;
9   }
10  nav ul {
11    list-style: none;
12  }
13  nav ul li {
14    display: inline-block;
15  }
16  nav ul li a {
17    color: white;
18    text-decoration: none;
19  }
20
21  /*# sourceMappingURL=style.css.map */
22
```

PARTE IV – PARTIALS E IMPORTS

À medida que os projetos CSS crescem, torna-se cada vez mais difícil gerir todos os estilos num único ficheiro. O SCSS resolve este problema através da utilização de partials e imports, permitindo dividir o código em vários ficheiros mais pequenos e organizados.

Os partials são ficheiros SCSS cujo nome começa por um sublinhado (_). Estes ficheiros não são compilados individualmente para CSS, servindo apenas para organizar partes específicas do código, como por exemplo:

- variáveis
- estilos do header
- estilos do footer
- botões
- layout geral

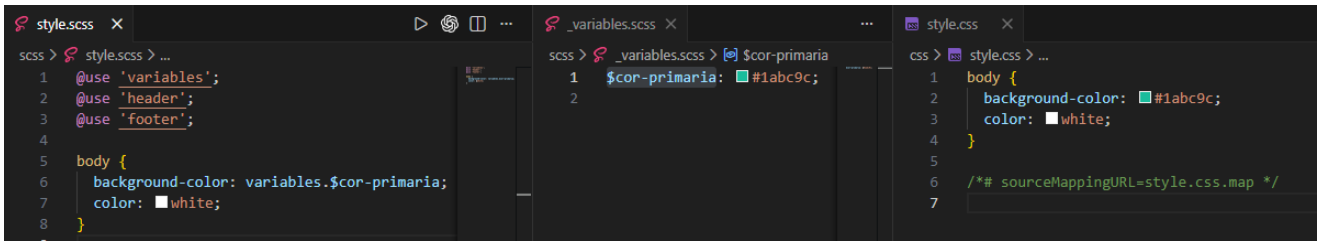
Através dos imports, podemos juntar todos esses ficheiros num único ficheiro principal, normalmente chamado style.scss, que será o responsável por gerar o ficheiro CSS final.

Esta abordagem permite:

- Melhor organização do código
- Maior facilidade de manutenção
- Reutilização de estilos
- Trabalho mais eficiente em equipa
- Projetos mais limpos e profissionais

O ficheiro style.scss deve funcionar apenas como ponto de entrada do projeto, sendo responsável por importar os vários partials e definir estilos globais. Os estilos específicos devem estar distribuídos pelos respetivos ficheiros parciais.

No exemplo seguinte será possível observar como dividir os estilos por ficheiros e como o SCSS os reúne num único CSS final durante o processo de compilação:



PARTE V – MIXINS

Os mixins são uma funcionalidade do SCSS que permite criar blocos de estilos reutilizáveis, evitando a repetição de código ao longo do projeto.

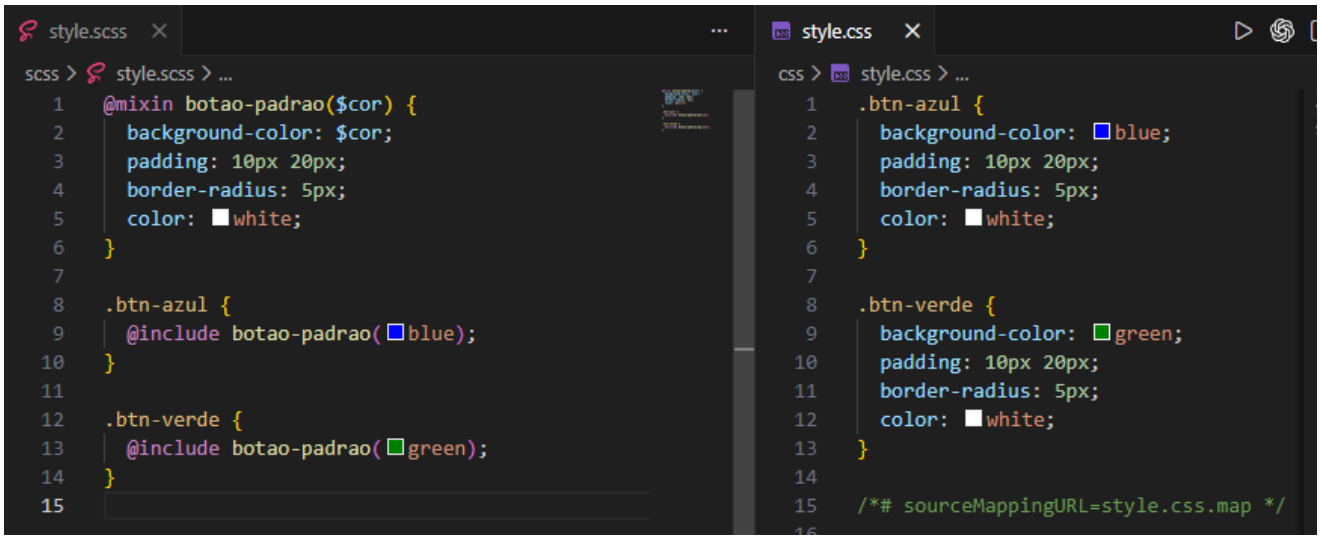
Em CSS tradicional, quando vários elementos partilham o mesmo conjunto de propriedades, é comum repetir o código em vários seletores. Com os mixins, é possível definir um conjunto de estilos uma única vez e reutilizá-lo sempre que necessário.

Os mixins podem ainda receber parâmetros, tornando-os mais flexíveis e adaptáveis a diferentes situações, como por exemplo cores, tamanhos ou espaçamentos.

A utilização de mixins permite:

- Reduzir duplicação de código
- Aumentar a consistência visual
- Facilitar a manutenção
- Tornar o código mais modular e organizado

No exemplo seguinte será demonstrado como criar e utilizar mixins para aplicar estilos reutilizáveis a vários elementos:



PARTE VI – EXTEND

O extend permite que um seletor herde os estilos de outro seletor, funcionando de forma semelhante a uma herança de estilos.

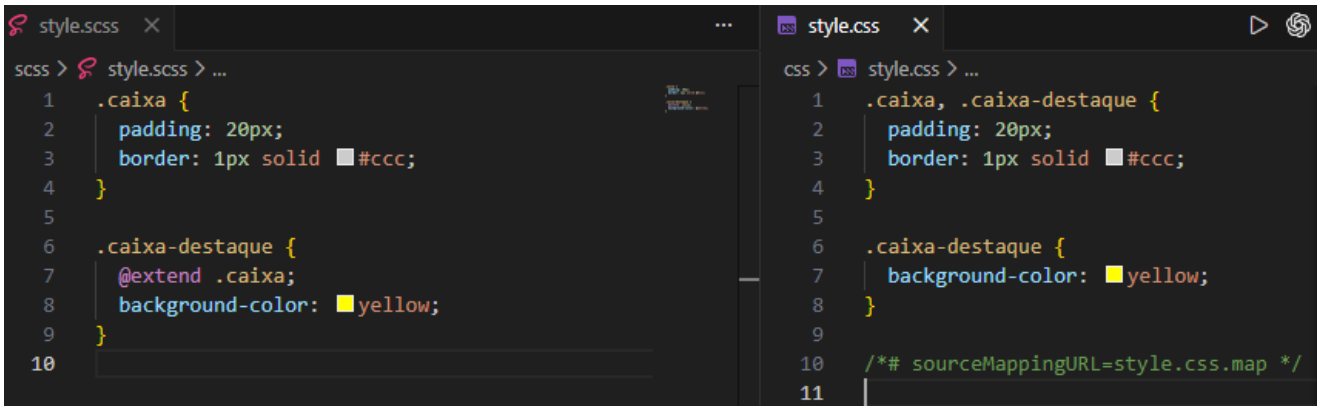
Ao utilizar @extend, um elemento passa a partilhar exatamente as mesmas propriedades de outro, sem necessidade de as reescrever. Esta funcionalidade é útil quando vários elementos têm uma base comum, mas apresentam pequenas variações.

Vantagens do extend:

- Evita duplicação de estilos
- Mantém o código limpo
- Facilita a consistência entre elementos semelhantes

O @extend deve ser usado com cuidado, pois pode gerar seletores CSS complexos e difíceis de interpretar. Em muitos casos, um mixin pode ser uma solução mais flexível.

No exemplo seguinte será possível observar como aplicar herança de estilos utilizando @extend:



PARTE VII – ESTRUTURAS DE CONTROLO EM SCSS

O SCSS permite utilizar estruturas de controlo, semelhantes às existentes em linguagens de programação, tornando o CSS mais dinâmico e poderoso.

@if e @else

As diretivas @if e @else permitem aplicar estilos consoante uma condição.
São úteis, por exemplo, para:

- Verificar valores
- Criar variações de tema
- Aplicar estilos condicionais

```scss
@if ($tema == claro) {
  background-color: white;
} @else {
  background-color: black;
}
```

@for

A diretiva @for permite criar repetições automáticas, sendo útil para gerar classes com padrões numéricos, como grelhas ou espaçamentos.

```scss
@for $i from 1 through 3 {
  .col-#{$i} {
    width: $i * 10%;
  }
}
```

```css
.col-1 {
  width: 10%;
}

.col-2 {
  width: 20%;
}

.col-3 {
  width: 30%;
}
```

@each

O @each permite percorrer listas ou mapas, aplicando estilos de forma automática.
É especialmente útil para:

- Criar classes de cores
- Gerar variantes de componentes
- Trabalhar com temas

```scss
@each $cor in red, green, blue {
  .texto-#{$cor} {
    color: $cor;
  }
}
```

```css
.texto-red {
  color: red;
}

.texto-green {
  color: green;
}

.texto-blue {
  color: blue;
}
```

PARTE VIII – COMENTÁRIOS

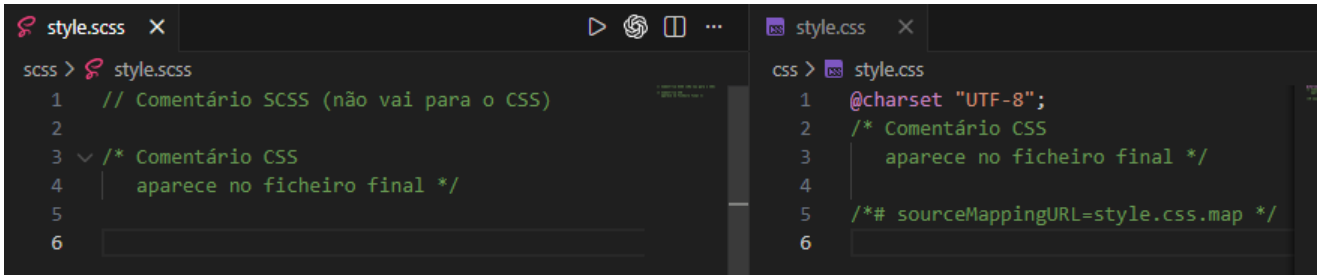
Tal como noutras linguagens, os comentários em SCSS servem para documentar o código, facilitar a sua leitura e ajudar na manutenção futura.

Existem dois tipos de comentários em SCSS:

- Comentários SCSS (//)
  São utilizados apenas durante o desenvolvimento e não aparecem no ficheiro CSS final.
- Comentários CSS (/* */)
  São mantidos no ficheiro CSS após a compilação e podem ser vistos no código final.

A correta utilização de comentários ajuda a:

- Explicar blocos de código
- Facilitar o trabalho em equipa
- Melhorar a compreensão do projeto



EXERCÍCIOS

1. Reformule o CSS do site que criamos anteriormente para SCSS.

Bom trabalho! ☺